UNIVERSIDAD TECNOLÓGICA DE CHIHUAHUA

Entornos Virtuales y Negocios Digitales

Programación de Aplicaciones Web Progresivas

Reporte de investigación de frameworks y librerías para desarrollar PWA

ALUMNO:

Ángel Iván López Delgado

Matrícula: 1122150052

Grupo: IEVND101M

PROFESOR:

Juan Carlos Bustamante Lozano

Chihuahua, Chih., a 8 de septiembre de 2026

1. Introducción

Una Progressive Web App (PWA) es una aplicación web que, mediante un conjunto de APIs estándar del navegador, se comporta como una aplicación nativa: se puede instalar en el dispositivo, funciona sin conexión, arranca desde un ícono propio y puede recibir notificaciones push. No es una tecnología aislada ni un framework, sino un modelo de aplicación que se apoya en tres piezas obligatorias:

- **HTTPS**, requisito de seguridad para que el navegador habilite las APIs sensibles.
- **Web App Manifest** (manifest.webmanifest), archivo JSON que declara nombre, íconos, color de tema y modo de despliegue, y que permite la instalación.
- **Service Worker**, un script que corre en un hilo aparte del documento y actúa como proxy de red programable: intercepta las peticiones, decide si responden desde caché o desde el servidor, y habilita el funcionamiento offline, la sincronización en segundo plano y las notificaciones push.

Escribir un Service Worker a mano es posible, pero rápidamente se vuelve un problema de ingeniería: hay que versionar cachés, invalidarlas cuando cambia el build, elegir una estrategia distinta por tipo de recurso y manejar el ciclo de vida de instalación y activación. De ahí que el ecosistema haya producido librerías y frameworks que resuelven ese trabajo repetitivo. Este reporte investiga cuatro de las opciones más usadas actualmente.

2. Objetivo

Investigar, describir y comparar cuatro frameworks y librerías vigentes para el desarrollo de Progressive Web Apps, identificando qué aporta cada uno al modelo PWA, sus ventajas y desventajas, y en qué escenario conviene elegir cada uno, con el fin de contar con un criterio técnico de selección para proyectos futuros.

3. Metodología

La investigación se realizó por revisión documental de las fuentes primarias de cada herramienta -documentación oficial, repositorios en GitHub y la documentación de referencia de web.dev y MDN Web Docs- durante septiembre de 2026. Para cada herramienta se registró: tipo (librería o framework), qué problema del modelo PWA

resuelve, cómo se integra en un proyecto, ventajas, desventajas y caso de uso recomendado. Al final se concentraron los hallazgos en una tabla comparativa y se emitió una recomendación.

Un punto de método importante: las cuatro herramientas **no compiten entre sí en el mismo nivel**. Workbox es una librería de bajo nivel; las otras tres son frameworks de aplicación que, de hecho, terminan usando Workbox o un equivalente por debajo. La comparación se hace entonces sobre el nivel de abstracción y el costo de adopción, no sobre "cuál genera mejores Service Workers".

4. Herramientas analizadas

4.1 Workbox

Tipo: librería (conjunto de módulos de JavaScript), mantenida por el equipo de Chrome de Google. Versión 7.x as of 2026-09 (https://developer.chrome.com/docs/workbox).

Qué resuelve. Workbox es la capa que abstrae el Service Worker. En lugar de escribir a mano los listeners de install, activate y fetch, se declaran rutas y estrategias de caché. Sus módulos principales son workbox-routing (asociar patrones de URL a estrategias), workbox-strategies (las estrategias listas: CacheFirst, NetworkFirst, StaleWhileRevalidate, NetworkOnly, CacheOnly), workbox-precaching (precarga y versionado automático del *app shell* al momento del build), workbox-expiration (límite de entradas y caducidad por caché), workbox-background-sync (reintento de peticiones fallidas cuando vuelve la conexión) y workbox-window (el lado del documento, para detectar y aplicar actualizaciones del Service Worker).

Ventajas.

- Es agnóstica del framework: funciona igual con React, Vue, Angular, Svelte o HTML plano.
- Control total y granular: se puede definir una estrategia distinta por tipo de recurso (por ejemplo CacheFirst para fuentes e imágenes, NetworkFirst para llamadas a la API, StaleWhileRevalidate para CSS y JS).
- Genera un manifiesto de precarga con *revisiones* (hashes), de modo que la invalidación de caché al desplegar una versión nueva es automática y no queda a criterio del programador.
- Es la base de facto: los plugins PWA de Vite, Angular, Create React App y otros la

usan por debajo, así que aprenderla sirve para todos.

Desventajas.

- Sigue exigiendo entender el ciclo de vida del Service Worker; no elimina la complejidad conceptual, solo la sintáctica.
- Requiere integrarse con el proceso de build (workbox-build, workbox-cli o un plugin del bundler) para que la precarga se genere correctamente.
- Configurar mal una estrategia puede dejar a los usuarios servidos desde una caché vieja; es el error más común al usarla.
- No aporta nada de la capa de UI ni de renderizado: es únicamente la capa de red y caché.

Caso de uso. Proyectos que ya tienen su propia arquitectura de front-end y solo necesitan agregar capacidades offline con control fino, o equipos que quieren entender y dominar el Service Worker en lugar de delegarlo por completo a un generador.

4.2 Next.js (con Serwist / next-pwa)

Tipo: framework de aplicación sobre React, desarrollado por Vercel. Serie 15-16 as of 2026-09 (https://nextjs.org/docs).

Qué resuelve. Next.js no es un framework "de PWA" en sentido estricto, pero cubre por sí solo dos de los tres requisitos y buena parte de los criterios de calidad que evalúa Lighthouse. Aporta renderizado en servidor (SSR) y generación estática (SSG), lo que da un primer pintado muy rápido; división automática del código por ruta; optimización de imágenes y fuentes; y, desde el App Router, una API de metadatos que genera el Web App Manifest desde un archivo app/manifest.ts tipado. El Service Worker se agrega mediante una librería del ecosistema: **Serwist** -sucesor mantenido de next-pwa, construido sobre Workbox- o una implementación propia registrada desde public/.

Ventajas.

- El rendimiento base es alto sin trabajo extra, y el rendimiento es parte del criterio de "progresiva": SSR/SSG mejoran directamente LCP y TTFB.
- Un solo proyecto cubre front-end, rutas de API (backend ligero) y PWA, lo que reduce la infraestructura del proyecto.
- Ecosistema y comunidad enormes; despliegue prácticamente sin configuración en Vercel u otras plataformas.

- El manifest tipado evita errores de dedo en el JSON y se versiona con el código.

Desventajas.

- La capa PWA es de terceros, no oficial: next-pwa quedó sin mantenimiento activo y hubo que migrar a Serwist, lo que ilustra el riesgo de depender del ecosistema.
- El SSR y el Service Worker conviven mal si no se configura con cuidado: cachear documentos HTML renderizados en servidor puede servir contenido obsoleto o, peor, contenido de otro usuario.
- Curva de aprendizaje real (App Router, Server Components, límites cliente/servidor) para un proyecto que quizá solo necesitaba ser una web instalable.
- Peso mayor que una solución mínima; para una app pequeña puede ser sobreingeniería.

Caso de uso. Aplicaciones de producto con contenido dinámico, SEO relevante y necesidad de backend ligero, donde la instalabilidad y el offline son una capa más y no el eje del proyecto.

4.3 Vite + vite-plugin-pwa

Tipo: plugin para el bundler Vite, mantenido por la comunidad (Anthony Fu / equipo de VitePWA). Versión 1.x as of 2026-09 (https://vite-pwa-org.netlify.app).

Qué resuelve. Es el camino más corto de un proyecto Vite a una PWA completa. Se instala como plugin en vite.config.js, se declara el manifest dentro de la configuración y el plugin se encarga del resto: genera el manifest.webmanifest, genera el Service Worker con Workbox y produce el manifiesto de precarga con todos los assets del build. Ofrece dos modos: generateSW, en el que el Service Worker se genera por completo desde la configuración, e injectManifest, en el que el desarrollador escribe su propio Service Worker y el plugin solo le inyecta la lista de archivos a precargar. Incluye además utilidades de UI para el flujo de actualización (el clásico aviso de "hay una versión nueva disponible") y soporte para probar el Service Worker en modo desarrollo.

Ventajas.

- Configuración mínima: una PWA funcional y auditable en Lighthouse con pocas líneas.
- Agnóstico del framework de UI: sirve igual para React, Vue, Svelte, Solid o Preact, siempre que el bundler sea Vite.

- Los dos modos permiten empezar declarativo y migrar a control total sin cambiar de herramienta.
- Build muy rápido, heredado de Vite y esbuild/Rollup.

Desventajas.

- Ata el proyecto a Vite; no aplica a proyectos con Webpack, Turbopack u otro bundler.
- Es una capa más sobre Workbox: cuando algo falla, hay que depurar en dos niveles (plugin y librería).
- No aporta SSR por sí mismo, así que el primer pintado en aplicaciones grandes depende del framework de UI que se use encima.
- Mantenimiento comunitario, sin el respaldo corporativo de Google o Vercel.

Caso de uso. Single Page Applications e interfaces de herramientas internas, dashboards o prototipos, donde el objetivo es tener offline e instalabilidad rápido y sin pelearse con la configuración.

4.4 Angular (@angular/pwa)

Tipo: framework de aplicación completo, desarrollado por Google. Serie 20+ as of 2026-09 (https://angular.dev/ecosystem/service-workers).

Qué resuelve. Angular es el único de los cuatro que trae soporte PWA **de primera parte**. Ejecutando ng add @angular/pwa el CLI agrega el paquete @angular/service-worker, crea el manifest.webmanifest con el juego completo de íconos, registra el Service Worker en el módulo raíz y genera el archivo de configuración ngsw-config.json. Ese archivo es el centro del modelo: ahí se declaran los assetGroups (recursos estáticos, con modo prefetch o lazy) y los dataGroups (respuestas de API, con estrategia freshness o performance, tamaño máximo y tiempo de vida). En tiempo de ejecución, el servicio SwUpdate permite detectar versiones nuevas y pedirle al usuario que recargue.

Ventajas.

- Solución oficial e integrada: versionado, actualización y rollback del Service Worker los maneja el propio framework, con un modelo de "versiones consistentes" que evita servir una mezcla de archivos de dos builds distintos.
- Configuración declarativa en un solo JSON, sin escribir lógica de Service Worker.

- Encaja en el resto del stack opinionado de Angular (TypeScript estricto, RxJS, inyección de dependencias, CLI), lo que hace muy predecibles los proyectos grandes y de equipo.
- Buen ajuste para aplicaciones empresariales de largo mantenimiento.

Desventajas.

- Curva de aprendizaje alta si no se conoce Angular: adoptar el framework completo solo por su PWA es desproporcionado.
- Menos flexible que Workbox directo: lo que ngsw-config.json no contempla es difícil de lograr sin salirse del modelo oficial.
- El Service Worker de Angular no se activa en ng serve; hay que compilar en producción y servir el dist/ para probarlo, lo que alarga el ciclo de prueba.
- Bundle inicial más pesado que las alternativas ligeras.

Caso de uso. Aplicaciones empresariales y de línea de negocio, con equipos grandes, ciclo de vida largo y necesidad de convenciones estrictas.

5. Tabla comparativa

Tabla 1. Comparativa de las herramientas analizadas.

Criterio	**Workbox**	**Next.js + Serwist**	**Vite + vite-plugin-pwa**	**Angular PWA**
Tipo	Librería	Framework + librería	Plugin de build	Framework
Respaldo	Google	Vercel + comunidad	Comunidad	Google
Nivel de abstracción	Bajo	Medio	Medio-alto	Alto
Soporte PWA oficial	Es la referencia	No, de terceros	No, de terceros	Sí, de primera parte
Curva de aprendizaje	Media	Alta	Baja	Alta
Control del Service Worker	Total	Alto	Configurable	Limitado al esquema
SSR / SEO	No aplica	Sí, nativo	No	Sí, con Angular SSR

Criterio	Workbox	Next.js + Serwist	Vite + vite-plugin-pwa	Angular PWA
Atadura tecnológica	Ninguna	React + Next	Vite	Angular
Mejor para	Control fino	Producto con SEO	SPA y dashboards	App empresarial

6. Conclusión

Las cuatro herramientas resuelven el mismo requisito técnico -un Service Worker correcto y un manifest válido- pero desde niveles de abstracción distintos, y esa es la variable real de decisión. **Workbox** es la base común y conviene entenderla aunque no se use directamente, porque las otras tres la usan o la imitan. **vite-plugin-pwa** ofrece la mejor relación esfuerzo/resultado para una aplicación de una sola página. **Next.js** es la opción cuando el proyecto necesita SEO, contenido dinámico y backend en el mismo repositorio, aceptando que su capa PWA depende del ecosistema. **Angular** es la única con soporte oficial de extremo a extremo, a cambio de adoptar el framework completo.

La recomendación práctica es elegir primero por las necesidades de la aplicación -renderizado, SEO, tamaño del equipo, ciclo de vida- y tratar la capa PWA como consecuencia de esa decisión, no al revés. Para un proyecto nuevo de tamaño mediano sin requisitos fuertes de SEO, la combinación Vite + vite-plugin-pwa es la que menos fricción impone; si el SEO importa, Next.js con Serwist; y en un entorno empresarial con Angular ya adoptado, @angular/pwa es la elección obvia.

7. Referencias

- Google Chrome Developers. *Workbox - Documentación oficial.* https://developer.chrome.com/docs/workbox (consultado 2026-09-08).
- MDN Web Docs. *Progressive web apps.* https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps (consultado 2026-09-08).
- web.dev. *Learn PWA.* https://web.dev/learn/pwa (consultado 2026-09-08).
- Vercel. *Next.js Documentation - Progressive Web Applications.* https://nextjs.org/docs/app/guides/progressive-web-apps (consultado 2026-09-08).
- Serwist. *Serwist Documentation.* https://serwist.pages.dev (consultado 2026-09-08).
- VitePWA. *Vite Plugin PWA - Guide.* https://vite-pwa-org.netlify.app (consultado

2026-09-08).

- Angular. *Angular service worker introduction.* https://angular.dev/ecosystem/service-workers (consultado 2026-09-08).

- W3C. *Web Application Manifest.* https://www.w3.org/TR/appmanifest (consultado 2026-09-08).